

TP 8 React Native :

Création du projet Firebase

Étape 1 - Créer le projet

- Aller sur <https://console.firebase.google.com>
- **Créer un projet**
- Désactiver Google Analytics

Étape 2 - Activer l'authentification

- **Authentication** → **Get started**
- Activer :
 - Email / Mot de passe
 - Google

Étape 3 - Firestore

- **Firestore Database**
- Mode **test**
- Collection : **todos**

Installation des dépendances

```
npx expo install firebase
npx expo install expo-auth-session expo-web-browser
npx expo install expo-camera expo-location expo-contacts
expo-notifications
```

Configuration Firebase

.env

```
EXPO_PUBLIC_FIREBASE_API_KEY=xxxx
EXPO_PUBLIC_FIREBASE_AUTH_DOMAIN=xxxx.firebaseio.com
EXPO_PUBLIC_FIREBASE_PROJECT_ID=xxxx
EXPO_PUBLIC_FIREBASE_APP_ID=xxxx

EXPO_PUBLIC_GOOGLE_WEB_CLIENT_ID=xxxxxx.apps.googleusercontent.com
```

Toutes les clés sont dans **.env** (non pushées sur GitHub)

/services/firebase.js

```
import { initializeApp } from "firebase/app";
import { getAuth } from "firebase/auth";
import { getFirestore } from "firebase/firestore";

const firebaseConfig = {
  apiKey: process.env.EXPO_PUBLIC_FIREBASE_API_KEY,
  authDomain: process.env.EXPO_PUBLIC_FIREBASE_AUTH_DOMAIN,
  projectId: process.env.EXPO_PUBLIC_FIREBASE_PROJECT_ID,
  appId: process.env.EXPO_PUBLIC_FIREBASE_APP_ID,
};

export const app = initializeApp(firebaseConfig);
export const auth = getAuth(app);
export const db = getFirestore(app);
```

Modification de AuthContext

```
import { createContext, useEffect, useState } from "react";
import { onAuthStateChanged, signOut } from "firebase/auth";
import { auth } from "../services/firebase";

export const AuthContext = createContext();

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const unsubscribe = onAuthStateChanged(auth, (u) => {
      setUser(u);
      setLoading(false);
    });

    return unsubscribe;
  }, []);

  const logout = () => signOut(auth);

  return (
    <AuthContext.Provider value={{ user, loading, logout }}>
      {children}
    </AuthContext.Provider>
  );
}
```

Authentication Email / Mot de passe

/screens/LoginScreen.js

```
import {
  View,
  Text,
  TextInput,
  TouchableOpacity,
  ActivityIndicator,
} from "react-native";
import { useContext, useEffect, useState } from "react";
import * as WebBrowser from "expo-web-browser";
import * as Google from "expo-auth-session/providers/google";
import {
  signInWithEmailAndPassword,
  createUserWithEmailAndPassword,
  GoogleAuthProvider,
  signInWithCredential,
} from "firebase/auth";

import { auth } from "../services/firebase";
import { ThemeContext } from "../context/ThemeContext";

WebBrowser.maybeCompleteAuthSession();

const WEB_CLIENT_ID = process.env.EXPO_PUBLIC_GOOGLE_WEB_CLIENT_ID;

export default function LoginScreen() {
  const { theme, toggleTheme } = useContext(ThemeContext);

  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);

  // 🔑 Google Auth (WEB - Expo)
  const [request, response, promptAsync] = Google.useAuthRequest({
    clientId: WEB_CLIENT_ID,
    responseType: "id_token",
    scopes: ["profile", "email"],
  });

  // 🔄 Handle Google response
  useEffect(() => {
    if (response?.type === "success") {

```

```

        const { id_token } = response.params;

        const credential = GoogleAuthProvider.credential(id_token);
        signInWithCredential(auth, credential).catch(() =>
            setError("Erreur Google Sign-In")
        );
    }
}, [response]);

const login = async () => {
    setError("");
    setLoading(true);
    try {
        await signInWithEmailAndPassword(auth, email, password);
    } catch (e) {
        setError("Email ou mot de passe incorrect");
    } finally {
        setLoading(false);
    }
};

const register = async () => {
    setError("");
    setLoading(true);
    try {
        await createUserWithEmailAndPassword(auth, email, password);
    } catch (e) {
        setError("Compte déjà existant ou mot de passe faible");
    } finally {
        setLoading(false);
    }
};

return (
    <View
        style={{
            flex: 1,
            backgroundColor: theme.background,
            justifyContent: "center",
            padding: 24,
        }}
    >
    <Text
        style={{
            fontSize: 28,
            fontWeight: "bold",

```

```

        color: theme.text,
        marginBottom: 20,
        textAlign: "center",
    }}
>
    Connexion
</Text>

{error !== "" && (
    <Text style={{ color: "red", marginBottom: 10, textAlign: "center" }}>
        {error}
    </Text>
)}

<TextInput
    placeholder="Email"
    placeholderTextColor="#999"
    value={email}
    onChangeText={setEmail}
    style={{
        borderWidth: 1,
        borderColor: "#ccc",
        borderRadius: 8,
        padding: 12,
        marginBottom: 12,
        color: theme.text,
    }}
/>

<TextInput
    placeholder="Mot de passe"
    placeholderTextColor="#999"
    value={password}
    onChangeText={setPassword}
    secureTextEntry
    style={{
        borderWidth: 1,
        borderColor: "#ccc",
        borderRadius: 8,
        padding: 12,
        marginBottom: 20,
        color: theme.text,
    }}
/>

{loading ? (

```

```

<ActivityIndicator />
) : (
  <>
    <TouchableOpacity
      onPress={login}
      style={{
        backgroundColor: theme.primary,
        padding: 14,
        borderRadius: 8,
        marginBottom: 10,
      }}
    >
      <Text style={{ color: "#fff", textAlign: "center" }}>
        Se connecter
      </Text>
    </TouchableOpacity>

    <TouchableOpacity
      onPress={register}
      style={{
        borderWidth: 1,
        borderColor: theme.primary,
        padding: 14,
        borderRadius: 8,
        marginBottom: 20,
      }}
    >
      <Text
        style={{
          color: theme.primary,
          textAlign: "center",
        }}
      >
        Créer un compte
      </Text>
    </TouchableOpacity>

    <TouchableOpacity
      disabled={!request}
      onPress={() => promptAsync()}
      style={{
        backgroundColor: "#DB4437",
        padding: 14,
        borderRadius: 8,
      }}
    >

```

```

        <Text style={{ color: "#fff", textAlign: "center" }}>
          Continuer avec Google
        </Text>
      </TouchableOpacity>
    </>
  )}

  <TouchableOpacity onPress={toggleTheme} style={{ marginTop: 30 }}>
    <Text style={{ textAlign: "center", color: theme.primary }}>
      Changer le thème
    </Text>
  </TouchableOpacity>
</View>
);
}

```

[/screens/ProfileScreen.js](#)

```

import { View, Text } from "react-native";
import { useContext } from "react";
import { AuthContext } from "../context/AuthContext";
import { ThemeContext } from "../context/ThemeContext";
import { SafeAreaView } from "react-native-safe-area-context";

export default function ProfileScreen() {
  const { user } = useContext(AuthContext);
  const { theme } = useContext(ThemeContext);

  return (
    <SafeAreaView style={{ flex: 1, backgroundColor: theme.background }}>
      <View style={{ padding: 20 }}>
        <Text style={{ fontSize: 24, color: theme.text }}>
          Profil utilisateur
        </Text>
        <Text style={{ color: theme.text }}>{user.email}</Text>
      </View>
    </SafeAreaView>
  );
}

```

Authentification Google (WEB UNIQUEMENT) – Lancée uniquement avec `npx expo start --web`

Android/iOS nécessitent configuration lourde (SHA-1, APK, Play Services)

Firestore – Gestion des tâches

/services/firestore.js

```
import { db } from "../firebase";
import {
  collection,
  addDoc,
  getDocs,
  deleteDoc,
  doc,
} from "firebase/firestore";

export async function fetchTodosFromFirestore(uid) {
  const snap = await getDocs(collection(db, "users", uid, "todos"));
  return snap.docs.map(d => ({ id: d.id, ...d.data() }));
}

export async function addTodoToFirestore(uid, todo) {
  await addDoc(collection(db, "users", uid, "todos"), {
    title: todo.title,
    completed: false,
    createdAt: Date.now(),
  });
}

export async function deleteTodoFromFirestore(uid, id) {
  await deleteDoc(doc(db, "users", uid, "todos", id));
}
```

Modification de useTodoStore pour intégrer firestore

```
import { create } from "zustand";
import { db } from "../services/database";
import {
  fetchTodosFromFirestore,
  addTodoToFirestore,
} from "../services/firestore";

export const useTodoStore = create((set, get) => ({
```



```

todos: [],

loadTodos: async (uid) => {
  //charger Firestore
  const remoteTodos = await fetchTodosFromFirestore(uid);

  //injecter SQLite
  db.transaction(tx => {
    tx.executeSql("DELETE FROM todos");
    remoteTodos.forEach(t =>
      tx.executeSql("INSERT INTO todos (title) VALUES (?)", [t.title])
    );
  });

  //charger SQLite vers Zustand
  db.transaction(tx => {
    tx.executeSql("SELECT * FROM todos", [], (_, res) => {
      set({ todos: res.rows._array });
    });
  });
},

addTodo: async (uid, title) => {
  // SQLite
  db.transaction(tx => {
    tx.executeSql("INSERT INTO todos (title) VALUES (?)", [title]);
  });

  // Firestore
  await addTodoToFirestore(uid, { title });

  // Refresh
  get().loadTodos(uid);
},
}}));

```

Modifier TodoListScreen

```

import { View, Text, FlatList, Button, TextInput } from "react-native";
import { useEffect, useContext, useState } from "react";
import { useTodoStore } from "../store/useTodoStore";
import { AuthContext } from "../context/AuthContext";
import AppBar from "../components/AppBar";

export default function TodoListScreen() {
  const { user } = useContext(AuthContext);
  const { todos, loadTodos, addTodo } = useTodoStore();

```

```

const [title, setTitle] = useState("");

useEffect(() => {
  if (user) {
    loadTodos(user.uid);
  }
}, [user]);

const handleAddTodo = () => {
  if (!title.trim()) return;

  addTodo(user.uid, title);
  setTitle("");
};

return (
  <View style={{ flex: 1 }}>
    <AppBar title="Mes tâches" />

    {/* Champ de saisie */}
    <View style={{ padding: 15 }}>
      <TextInput
        placeholder="Nouvelle tâche..."
        value={title}
        onChangeText={setTitle}
        style={{
          borderWidth: 1,
          borderColor: "#ccc",
          padding: 10,
          borderRadius: 6,
          marginBottom: 10,
        }}
      />

      <Button title="Ajouter la tâche" onPress={handleAddTodo} />
    </View>

    {/* Liste */}
    <FlatList
      data={todos}
      keyExtractor={(i) => i.id.toString()}
      renderItem={({ item }) => (
        <Text style={{ padding: 15, fontSize: 16 }}>
          • {item.title}
        </Text>
      )}
    />
  </View>
);

```

```

    })
  />
</View>
);
}

```

Modifier le contenu de HomeScreen.js

```

import {
  View,
  Text,
  FlatList,
  TouchableOpacity,
  Modal,
  TextInput,
} from "react-native";
import { useContext, useEffect, useState } from "react";
import { ThemeContext } from "../context/ThemeContext";
import { AuthContext } from "../context/AuthContext";
import { collection, addDoc, getDocs } from "firebase/firestore";
import { db } from "../services/firebase";
import { SafeAreaView } from "react-native-safe-area-context";

export default function HomeScreen() {
  const { theme, toggleTheme } = useContext(ThemeContext);
  const { user, logout } = useContext(AuthContext);

  const [todos, setTodos] = useState([]);
  const [modalVisible, setModalVisible] = useState(false);
  const [newTodo, setNewTodo] = useState("");

  const loadTodos = async () => {
    const snap = await getDocs(collection(db, "todos"));
    setTodos(snap.docs.map(d => ({ id: d.id, ...d.data() })));
  };

  useEffect(() => {
    loadTodos();
  }, []);

  const addTodo = async () => {
    if (!newTodo.trim()) return;

    await addDoc(collection(db, "todos"), {
      title: newTodo,
      userId: user.uid,
    });
  };

```

```

        createdAt: new Date(),
    });

    setNewTodo("");
    setModalVisible(false);
    loadTodos();
};

return (
    <SafeAreaView style={{ flex: 1, backgroundColor: theme.background }}>
        <View style={{ padding: 16 }}>
            <Text style={{ color: theme.text, fontSize: 26, fontWeight: "bold" }}>
                Mes tâches
            </Text>

            <TouchableOpacity onPress={toggleTheme}>
                <Text style={{ color: theme.primary }}>Changer thème</Text>
            </TouchableOpacity>

            <TouchableOpacity onPress={logout}>
                <Text style={{ color: "red" }}>Déconnexion</Text>
            </TouchableOpacity>

            <TouchableOpacity
                style={{
                    marginVertical: 15,
                    backgroundColor: theme.primary,
                    padding: 12,
                    borderRadius: 8,
                }}
                onPress={() => setModalVisible(true)}
            >
                <Text style={{ color: "#fff", textAlign: "center" }}>
                    + Ajouter une tâche
                </Text>
            </TouchableOpacity>

            <FlatList
                data={todos}
                keyExtractor={item => item.id}
                renderItem={({ item }) => (
                    <View
                        style={{
                            backgroundColor: theme.card,
                            padding: 12,
                            borderRadius: 8,

```

```

        marginBottom: 10,
      }}
    >
    <Text style={{ color: theme.text }}>{item.title}</Text>
  </View>
)}
/>
</View>

{/* MODAL FORM */}
<Modal visible={modalVisible} animationType="slide" transparent>
  <View
    style={{
      flex: 1,
      backgroundColor: "rgba(0,0,0,0.5)",
      justifyContent: "center",
      padding: 20,
    }}
  >
    <View
      style={{
        backgroundColor: theme.background,
        padding: 20,
        borderRadius: 10,
      }}
    >
      <Text style={{ color: theme.text, fontSize: 18 }}>
        Nouvelle tâche
      </Text>

      <TextInput
        placeholder="Titre de la tâche"
        value={newTodo}
        onChangeText={setNewTodo}
        style={{
          borderWidth: 1,
          borderColor: "#ccc",
          marginVertical: 10,
          padding: 10,
          borderRadius: 6,
          color: theme.text,
        }}
      />

      <TouchableOpacity
        onPress={addTodo}

```

```

        style={{
          backgroundColor: theme.primary,
          padding: 10,
          borderRadius: 6,
        }}
      >
        <Text style={{ color: "#fff", textAlign: "center" }}>
          Ajouter
        </Text>
      </TouchableOpacity>

      <TouchableOpacity onPress={() => setModalVisible(false)}>
        <Text style={{ textAlign: "center", marginTop: 10 }}>
          Annuler
        </Text>
      </TouchableOpacity>
    </View>
  </View>
</Modal>
</SafeAreaView>
);
}

```

Modifier AppStack

```

import { NavigationContainer } from "@react-navigation/native";
import { createNativeStackNavigator } from "@react-navigation/native-stack";
import { useContext } from "react";
import { AuthContext } from "../context/AuthContext";
import LoginScreen from "../screens/LoginScreen";
import AppDrawer from "../AppDrawer";

const Stack = createNativeStackNavigator();

export default function AppStack() {
  const { user, loading } = useContext(AuthContext);
  if (loading) return null;

  return (
    <NavigationContainer>
      <Stack.Navigator screenOptions={{ headerShown: false }}>
        {!user ? (
          <Stack.Screen name="Login" component={LoginScreen} />
        ) : (
          <Stack.Screen name="App" component={AppDrawer} />
        )}
      </Stack.Navigator>
    </NavigationContainer>
  );
}

```

```

    </Stack.Navigator>
  </NavigationContainer>
);
}

```

Modifier AppDrawer

```

import { createDrawerNavigator } from "@react-navigation/drawer";
import HomeScreen from "../screens/HomeScreen";
import ProfileScreen from "../screens/ProfileScreen";

const Drawer = createDrawerNavigator();

export default function AppDrawer() {
  return (
    <Drawer.Navigator screenOptions={{ headerShown: true }}>
      <Drawer.Screen name="Mes tâches" component={HomeScreen} />
      <Drawer.Screen name="Profil" component={ProfileScreen} />
    </Drawer.Navigator>
  );
}

```

Modifier ThemeContext

```

import React, { createContext, useEffect, useState } from "react";
import AsyncStorage from "@react-native-async-storage/async-storage";

export const ThemeContext = createContext();

const themes = {
  light: {
    background: "#ffffff",
    text: "#111111",
    card: "#f2f2f2",
    primary: "#2f80ed",
  },
  dark: {
    background: "#121212",
    text: "#ffffff",
    card: "#1e1e1e",
    primary: "#2f80ed",
  },
};

const STORAGE_KEY = "APP_THEME";

```

```

export function ThemeProvider({ children }) {
  const [mode, setMode] = useState("light");
  const [ready, setReady] = useState(false);

  // Charger le thème au démarrage
  useEffect(() => {
    const loadTheme = async () => {
      const storedTheme = await AsyncStorage.getItem(STORAGE_KEY);
      if (storedTheme) {
        setMode(storedTheme);
      }
      setReady(true);
    };

    loadTheme();
  }, []);

  // Changer + sauvegarder le thème
  const toggleTheme = async () => {
    const newMode = mode === "light" ? "dark" : "light";
    setMode(newMode);
    await AsyncStorage.setItem(STORAGE_KEY, newMode);
  };

  // Empêche un flash blanc/noir au lancement
  if (!ready) return null;

  return (
    <ThemeContext.Provider
      value={{
        mode,
        theme: themes[mode],
        toggleTheme,
      }}
    >
      {children}
    </ThemeContext.Provider>
  );
}

```

Modifier App.js

```

import { ThemeProvider } from "../context/ThemeContext";
import { AuthProvider } from "../context/AuthContext";
import AppStack from "../navigation/AppStack";
import { SafeAreaProvider } from "react-native-safe-area-context";

```



```
export default function App() {
  return (
    <SafeAreaProvider>
      <ThemeProvider>
        <AuthProvider>
          <AppStack />
        </AuthProvider>
      </ThemeProvider>
    </SafeAreaProvider>
  );
}
```

Accès aux fonctionnalités natives avec Expo

NativeFeaturesScreen (menu des fonctionnalités)

/screens/[NativeFeaturesScreen.js](#)

```
import { View, Text, TouchableOpacity } from "react-native";

import AppBar from "../components/AppBar";

export default function NativeFeaturesScreen({ navigation }) {

  return (

    <View style={{ flex: 1 }}>

      <AppBar title="Fonctionnalités natives" />

      <TouchableOpacity onPress={() => navigation.navigate("Caméra")}>

        <Text>📷 Caméra</Text>

      </TouchableOpacity>

      <TouchableOpacity onPress={() => navigation.navigate("Localisation")}>

        <Text>📍 Géolocalisation</Text>

      </TouchableOpacity>

      <TouchableOpacity onPress={() => navigation.navigate("Contacts")}>

        <Text>👥 Contacts</Text>

      </TouchableOpacity>

    </View>
  );
}
```

```

    </TouchableOpacity>

    <TouchableOpacity onPress={() => navigation.navigate("Notifications")}>

      <Text>🔔 Notifications</Text>

    </TouchableOpacity>

  </View>

);
}

```

/navigation/[NativeStack.js](#)

```

import { createNativeStackNavigator } from "@react-navigation/native-stack";

import NativeFeaturesScreen from "../screens/NativeFeaturesScreen";

import CameraScreen from "../screens/CameraScreen";

import LocationScreen from "../screens/LocationScreen";

import ContactsScreen from "../screens/ContactsScreen";

import NotificationsScreen from "../screens/NotificationsScreen";

const Stack = createNativeStackNavigator();

export default function NativeStack() {

  return (

    <Stack.Navigator screenOptions={{ headerShown: false }}>

      <Stack.Screen name="Fonctionnalités" component={NativeFeaturesScreen} />

      <Stack.Screen name="Caméra" component={CameraScreen} />

      <Stack.Screen name="Localisation" component={LocationScreen} />

      <Stack.Screen name="Contacts" component={ContactsScreen} />

      <Stack.Screen name="Notifications" component={NotificationsScreen} />

    </Stack.Navigator>

  );
}

```

```
}
```

/screens/[CameraScreen.js](#)

```
import { CameraView, useCameraPermissions } from "expo-camera";

import { View, Text, Button, StyleSheet } from "react-native";

import { useRef } from "react";

import AppBar from "../components/AppBar";

export default function CameraScreen({ navigation }) {

  const [permission, requestPermission] = useCameraPermissions();

  const cameraRef = useRef(null);

  if (!permission) return <Text>Chargement...</Text>;

  // Permission refusée définitivement

  if (!permission.granted && !permission.canAskAgain) {

    return (

      <View style={styles.center}>

        <AppBar title="Caméra" back />

        <Text>Accès caméra refusé</Text>

        <Button title="Ouvrir les paramètres" onPress={() => Linking.openSettings()} />

      </View>

    );

  }

  // Permission non accordée
```

```

if (!permission.granted) {

  return (

    <View style={styles.center}>

      <AppBar title="Caméra" back />

      <Text>Accès caméra requis</Text>

      <Button title="Autoriser" onPress={requestPermission} />

    </View>

  );
}

return (

  <View style={{ flex: 1 }}>

    <AppBar title="Caméra" back />

    <CameraView

      ref={cameraRef}

      style={{ flex: 1 }}

    />

    <View style={styles.controls}>

      <Button title="⬅ Retour" color="gray" onPress={() => navigation.goBack()}

    />

    </View>

  </View>

);
}

const styles = StyleSheet.create({

```

```
center: {

  flex: 1,

  justifyContent: "center",

  alignItems: "center",

},

controls: {

  padding: 15,

  flexDirection: "row",

  justifyContent: "space-around",

  backgroundColor: "#fff",

},

});
```

/screens/[LocationScreen.js](#)

```
import * as Location from "expo-location";

import { View, Text, Button } from "react-native";

import { useState } from "react";

import AppBar from "../components/AppBar";

export default function LocationScreen() {

  const [location, setLocation] = useState(null);

  const getLocation = async () => {

    const { status } = await Location.requestForegroundPermissionsAsync();

    if (status !== "granted") return;

    const loc = await Location.getCurrentPositionAsync({});
```

```

        setLocation(loc.coords);

    };

    return (

        <View>

            <AppBar title="Localisation" back />

            <Button title="Obtenir position" onPress={getLocation} />

            {location && (

                <Text>

                    Lat: {location.latitude} | Lng: {location.longitude}

                </Text>

            )}

        </View>

    );

}

```

/screens/[ContactsScreen.js](#)

```

import * as Contacts from "expo-contacts";

import { View, Text, Button, FlatList } from "react-native";

import { useState } from "react";

import AppBar from "../components/AppBar";

export default function ContactsScreen() {

    const [contacts, setContacts] = useState([]);

    const loadContacts = async () => {

        const { status } = await Contacts.requestPermissionsAsync();
    }

```

```

    if (status !== "granted") return;

    const { data } = await Contacts.getContactsAsync({

      fields: [Contacts.Fields.Name],

    });

    setContacts(data);

  };

  return (

    <View>

      <AppBar title="Contacts" back />

      <Button title="Charger contacts" onPress={loadContacts} />

      <FlatList

        data={contacts}

        keyExtractor={(item) => item.id}

        renderItem={({ item }) => <Text>{item.name}</Text>}

      />

    </View>

  );
}

```

/screens/[NotificationsScreen.js](#)

```

import { View, Text, Button, StyleSheet } from "react-native";

import * as Notifications from "expo-notifications";

import AppBar from "../components/AppBar";

```

```
// Gestionnaire obligatoire (sinon Android n'affiche rien)

Notifications.setNotificationHandler({

  handleNotification: async () => ({

    shouldShowAlert: true,

    shouldPlaySound: true,

    shouldSetBadge: false,

  }),

});

export default function NotificationsScreen() {

  const requestPermission = async () => {

    const { status } = await Notifications.requestPermissionsAsync();

    alert(

      status === "granted"

        ? "Permission accordée"

        : "Permission refusée"

    );

  };

  const sendTestNotification = async () => {

    await Notifications.scheduleNotificationAsync({

      content: {

        title: "🔔 Notification locale",

        body: "Ceci est une notification locale de test",

      },

      trigger: null, // immédiat

    });

  };

}
```



```

};

return (

  <View style={{ flex: 1 }}>

    <AppBar title="Notifications" back />

    <View style={styles.container}>

      <Text style={styles.title}>Gestion des notifications locales</Text>

      <Button title="Demander permission" onPress={requestPermission} />

      <View style={styles.spacer} />

      <Button

        title="Notification immédiate"

        onPress={sendTestNotification}

      />

      <View style={styles.spacer} />

    </View>

  </View>

);
}

const styles = StyleSheet.create({

  container: {

```

```
padding: 20,  
  
},  
  
title: {  
  
  fontSize: 18,  
  
  marginBottom: 20,  
  
  fontWeight: "bold",  
  
  textAlign: "center",  
  
},  
  
spacer: {  
  
  height: 15,  
  
},  
  
});
```

Ajouter **NativeStack** dans le **AppDrawer**

Créer un item de menu "**Fonctionnalités natives**"

Ajouter un lien de retour au niveau de AppBar